Aufgabe 3: Lieferkette

Teilnahme-ID: 80513

Bearbeiter/-in dieser Aufgabe:
Peter Staykov

11. April 2026

Inhaltsverzeichnis

1 Lösungsidee **1**

2 Umsetzung **1**
2.1 Vorberechnung der Distanzen 1
2.2 Berechnung des Hauptplans 2
2.3 Streiksimulation 2

3 Verwendete Werkzeuge und Vorgehensweise **2**
3.1 Werkzeuge 2
3.2 Vorgehensweise 2

4 Beispiele **2**

5 Quellcode **3**

1 Lösungsidee

Das Ziel ist die Ermittlung eines zeiteffizienten Hauptplans sowie robuster Alternativpläne für den Fall eines Werksstreiks. Die Lösung basiert auf der Abbildung der logistischen Randbedingungen in einer spezialisierten Graphstruktur. Die Straßenkarte wird als gewichteter, ungerichteter Graph dargestellt. Darauf aufbauend wird ein logischer Schichtgraph konzipiert, in dem die Knoten die Werke repräsentieren und Übergänge nur zwischen aufeinanderfolgenden Arbeitsschritten i und $i+1$ betrachtet werden. Dies erzwingt die strikte Einhaltung der Produktionsreihenfolge $1, \ldots, m$. Die Gewichtung der Übergänge mit den kürzesten Fahrzeiten stellt sicher, dass jede Pfadsuche automatisch die Gesamtfahrzeit zum Zielpunkt T optimiert.

Da ein Streik erst bei Ankunft am Werk bemerkt wird, muss das Modell Flexibilität innerhalb einer Schicht erlauben. Im Streikfall wird ein Übergang zu einem alternativen Werk desselben Arbeitsschritts gesucht. Dabei ist es essenziell, dass der bereits zurückgelegte Weg bis zum streikenden Werk als unveränderlich betrachtet wird und die Route erst ab diesem Standort neu berechnet wird.

2 Umsetzung

Die technische Realisierung erfolgt modular durch eine Kombination aus Kürzeste-Wege-Suche und Dynamischer Programmierung (DP):

2.1 Vorberechnung der Distanzen

Zunächst werden die kürzesten Fahrzeiten im gesamten Straßennetz berechnet. Ein mit einem Minimum-Heap optimierter Dijkstra-Algorithmus ermittelt für den Startpunkt S, den Zielpunkt T sowie für jedes Werk die minimalen Distanzen zu allen anderen relevanten Knoten. Diese Distanzmatrix (`dist_map`) dient im Folgenden als schnelles Nachschlagewerk für die Kantengewichte zwischen den Schichten.

2.2 Berechnung des Hauptplans

Auf Basis der vorberechneten Distanzen wird der optimale Hauptplan mittels Dynamischer Programmierung (`find_best_sequence`) ermittelt. Für jede Stufe k wird für alle zugehörigen Werke der kostengünstigste Pfad von den Werken der vorherigen Stufe $k-1$ berechnet und gespeichert (`dp_cur`). Am Ende wird das Werk der letzten Stufe gewählt, welches in Summe mit der Fahrt zum Ziel T die minimalen Gesamtkosten aufweist. Durch Zurückverfolgen der Entscheidungen (`prev_choice`) wird die optimale Abfolge der Werke rekonstruiert.

2.3 Streiksimulation

Für jedes Werk im Hauptplan wird ein Streikszenario durchgespielt. Da die Information über den Streik erst vor Ort vorliegt, bleiben die akkumulierten Kosten vom Start bis zum bestreikten Werk bestehen (`prefix_costs`). Von diesem bestreikten Werk aus werden alle alternativen Werke der gleichen Stufe betrachtet. Die neuen Gesamtkosten ergeben sich aus der Summe von:

1. Den Kosten bis zum bestreikten Werk.
2. Der Distanz vom bestreikten Werk zur Alternative.
3. Den optimalen Restkosten (`_compute_tail_from_node`) von der Alternative über alle ausstehenden Stufen bis zum Ziel T.

Das Minimum dieser Gesamtkosten bestimmt die beste Ausweichroute. Auch hierbei wird nicht nur die Zeit, sondern auch die exakte neue Abfolge der Werke gespeichert.

3 Verwendete Werkzeuge und Vorgehensweise

3.1 Werkzeuge

Es besteht eine vollständige Eigenimplementierung der Graphenlogik und des Dijkstra-Algorithmus in Python ohne externe Bibliotheken. Eine KI wurde punktuell für die Formulierung und die sprachliche Korrektur der Dokumentation genutzt.

3.2 Vorgehensweise

Die Entwicklung startete mit der Anforderung, die Schrittfolge $1 \dots m$ zwingend einzuhalten. Dies führte zur Entkopplung von Wegfindung (Dijkstra) und Sequenzplanung (Dynamische Programmierung). Die größte Herausforderung war die Modellierung des Streiks. Die Logik wurde so konzipiert, dass nicht der gesamte Plan ab S neu berechnet wird, sondern das System erst ab dem Problemknoten flexibel reagiert. Da gemäß Aufgabenstellung höchstens ein Streik gleichzeitig auftritt, bleibt der Hauptplan bis zum betroffenen Werk die garantiert beste Wahl.

4 Beispiele

```
MOEGLICH
15

9
S [3 1] [5 2] T
11
3 [2 1] [5 2] T
15
5 [4 2] T
```

Listing 1: lieferung_00.txt Output

```
MOEGLICH
40

26
S [1 1] [4 2] [7 3] T
40
```

```
1 [2 1] [4 2] [7 3] T
37
4 [5 2] [8 3] T
38
7 [6 3] T
```

Listing 2: lieferung_01.txt Output

```
MOEGLICH
188

122
S [1 1] [2 2] [4 3] [10 4] [7 5] T
138
1 [5 1] [3 2] [4 3] [10 4] [7 5] T
157
2 [3 2] [4 3] [10 4] [7 5] T
188
4 [11 3] [10 4] [7 5] T
174
10 [6 4] [7 5] T
122
7 [13 5] T
```

Listing 3: lieferung_02.txt Output

```
UNMOEGLICH
```

Listing 4: lieferung_03.txt Output

5 Quellcode

```python
import heapq

def dijkstra_all(adj, source):
    INF = float('inf')
    dist = {n: INF for n in adj.keys()}
    prev = {n: None for n in adj.keys()}
    dist[source] = 0
    heap = [(0, source)]

    while heap:
        d_u, u = heapq.heappop(heap)
        if d_u != dist[u]:
            continue
        for v, w in adj.get(u, []):
            nd = d_u + w
            if nd < dist.get(v, INF):
                dist[v] = nd
                prev[v] = u
                heapq.heappush(heap, (nd, v))
    return dist, prev

def find_best_sequence(stages, dist_map, source='S', target='T'):
    m = max(stages.values())
    stage_nodes = {s: [] for s in range(1, m + 1)}
    for n, s in stages.items():
        stage_nodes[s].append(n)

    INF = float('inf')
    dp_prev = {p: dist_map[source].get(p, INF) for p in stage_nodes[1]}
    prev_choice = {(1, p): source for p in stage_nodes[1]}

    for k in range(2, m + 1):
        dp_cur = {}
        for p2 in stage_nodes[k]:
            best, best_prev = INF, None
            for p1, cost1 in dp_prev.items():
                tot = cost1 + dist_map[p1].get(p2, INF)
                if tot < best:
                    best, best_prev = tot, p1
```

```
            if best < INF:
                dp_cur[p2], prev_choice[(k, p2)] = best, best_prev
        dp_prev = dp_cur

    best_total, best_last = INF, None
    for p, cost in dp_prev.items():
        tot = cost + dist_map[p].get(target, INF)
        if tot < best_total:
            best_total, best_last = tot, p

    if best_last is None:
        return INF, []

    seq = [None] * m
    cur = best_last
    for k in range(m, 0, -1):
        seq[k - 1] = cur
        cur = prev_choice.get((k, cur))
    return best_total, seq

def compute_tail_from_node(stages, start_node, start_stage, dist_map):
    m = max(stages.values())
    stage_nodes = {s: [] for s in range(1, m + 1)}
    for n, s in stages.items():
        stage_nodes[s].append(n)

    INF = float('inf')
    dp_prev = {start_node: 0}
    prev_choice = {}

    for k in range(start_stage + 1, m + 1):
        dp_cur = {}
        for p2 in stage_nodes[k]:
            best, best_prev = INF, None
            for p1, cost1 in dp_prev.items():
                tot = cost1 + dist_map[p1].get(p2, INF)
                if tot < best:
                    best, best_prev = tot, p1
            if best < INF:
                dp_cur[p2], prev_choice[(k, p2)] = best, best_prev
        dp_prev = dp_cur

    best_total, best_last = INF, None
    for p, cost in dp_prev.items():
        tot = cost + dist_map[p].get('T', INF)
        if tot < best_total:
            best_total, best_last = tot, p

    if best_last is None:
        return INF, []

    seq = [None] * (m - start_stage + 1)
    seq[-1] = best_last
    cur = best_last
    for k in range(m, start_stage, -1):
        prevn = prev_choice.get((k, cur))
        seq[k - start_stage - 1] = prevn
        cur = prevn
    seq[0] = start_node
    return best_total, seq

def simulate_strike_times(adj, stages, source="S", target="T"):
    sources = {source, target} | set(stages.keys())
    dist_map = {s: dijkstra_all(adj, s)[0] for s in sources}

    base_total, base_seq = find_best_sequence(stages, dist_map, source, target)
    if base_total == float('inf'):
        return None

    prefix_costs = [dist_map[source].get(base_seq[0], float('inf'))]
    for i in range(1, len(base_seq)):
        d = dist_map[base_seq[i - 1]].get(base_seq[i], float('inf'))
        prefix_costs.append(prefix_costs[-1] + d)
```

```
113
    results = {'base_total': base_total, 'base_seq': base_seq}
115 stage_nodes = {s: [] for s in range(1, max(stages.values()) + 1)}
    for n, s in stages.items():
117     stage_nodes[s].append(n)

119 for i, struck in enumerate(base_seq):
        stage_index = stages[struck]
121     candidates = [v for v in stage_nodes[stage_index] if v != struck]
        best_alt, best_seq = float('inf'), []
123
        for b in candidates:
125         d_struck_to_b = dist_map[struck].get(b, float('inf'))
            if d_struck_to_b == float('inf'):
127             continue

129         tail_cost, tail_seq = compute_tail_from_node(stages, b, stage_index, dist_map)
            total = prefix_costs[i] + d_struck_to_b + tail_cost
131
            if total < best_alt:
133             best_alt = total
                best_seq = [struck] + tail_seq
135
        results[struck] = {'best_time': best_alt, 'alt_sequence': best_seq}
137
    return results
```

Listing 5: Quellcode